# Agni-Drishti Backend Documentation

Generated from the current backend structure. This backend is a FastAPI/PostgreSQL/PostGIS application for thermal-anomaly and wildfire monitoring.

## 1. Backend Purpose

- Accepts VNF satellite detection CSV files.
- Cleans, spatially clusters, and temporally analyzes thermal detections.
- Builds persistent thermal objects and classifies them with an XGBoost model.
- Fetches NASA FIRMS detections and matches them to known thermal objects.
- Creates alerts for unmatched, repeated, or abnormal activity.
- Exposes object, monitoring, VNF, and dashboard APIs.

## 2. Running the Backend

```
pip install -r requirements.txt
uvicorn app.main:app --reload
```

API documentation: http://127.0.0.1:8000/docs

The application expects a .env file containing DATABASE_URL and FIRMS_MAP_KEY. The database must support PostgreSQL/PostGIS features such as ST_DWithin and ST_Distance.

## 3. Application Flow

```
Client -> FastAPI routes -> controllers -> services -> pipeline/external APIs ->
PostgreSQL/PostGIS
```

VNF upload flow: validate CSV, ingest raw detections, match candidates, run spatial and temporal processing, generate features, classify objects, synchronize thermal objects, and recalculate profiles.

FIRMS flow: create monitoring run, fetch NASA FIRMS data, filter to India, deduplicate, match with thermal objects, insert new detections, create alerts, and update run statistics.

## 4. API Endpoints

### API surface

| File | Purpose |
|---|---|
| GET / | Health check from app/main.py. |
| GET /objects/ | Lists thermal objects with optional class_name, min_confidence, and limit filters. |
| GET /monitoring/summary | Returns monitoring summary data. |
| GET /monitoring/recent | Returns recent FIRMS monitoring detections. |
| GET /monitoring/unmatched | Returns unmatched detections. |
| POST /vnf/upload | Accepts a VNF CSV upload and runs incremental or bootstrap processing. |
| POST /monitoring/runs/trigger | Starts a FIRMS monitoring run; day_range is between 1 and 10. |
| GET /monitoring/runs/latest | Returns the latest monitoring run. |
| GET /monitoring/runs | Returns monitoring history; limit is between 1 and 100. |
| GET /dashboard/summary | Dashboard summary metrics. |
| GET /dashboard/classes | Thermal-object class statistics. |

| File | Purpose |
|---|---|
| GET /dashboard/objects | Paginated dashboard object list with optional classification. |
| GET /dashboard/objects/{object_uid} | Details for one thermal object. |
| GET /dashboard/firms/recent | Recent FIRMS detections with optional matched_only filter. |
| GET /dashboard/alerts | Alerts with optional alert_type filter. |
| GET /dashboard/monitoring | Monitoring dashboard data. |

# 5. Core and Application Files

## Core

| File | Purpose |
|---|---|
| app/main.py | Creates the FastAPI app, imports models, creates tables at startup, registers routers, and exposes GET /. |
| app/core/config.py | Loads environment settings and paths for the database, VNF data, boundary GeoJSON, and classifier artifact. |
| app/core/database.py | Creates the SQLAlchemy engine, session factory, declarative Base, and get_db dependency. |
| app/core/logger.py | Configures application logging and exposes get_logger(). |
| app/controllers/object_controller.py | Implements thermal-object API operations. |
| app/controllers/monitoring_controller.py | Implements monitoring summary, recent, and unmatched API operations. |
| app/schemas/thermal_object_schema.py | Defines the Pydantic ThermalObjectResponse API response schema with ORM support. |

# 6. Database Models

## Models

| File | Purpose |
|---|---|
| app/models/vnf_detection_model.py | Stores raw VNF detections, measurements, processing state, object UID, and ingestion batch. |
| app/models/thermal_object_model.py | Stores persistent thermal objects, classifications, confidence, metrics, radius, and fingerprint. |
| app/models/firms_detection_model.py | Stores NASA FIRMS observations, FRP, location, matching data, and persistent-source status. |
| app/models/alert_model.py | Stores alert type, severity, message, related detections/objects, location, and resolution state. |
| app/models/monitoring_run_model.py | Tracks run status, timing, retries, counters, and errors for FIRMS monitoring. |
| app/models/__init__.py | Empty package initializer for database models. |

# 7. VNF Pipeline

## Pipeline modules

| File | Purpose |
|---|---|
| app/pipeline/vnf/preprocessing.py | Parses dates and removes invalid values, coordinates, dates, and out-of-area records. |
| app/pipeline/vnf/spatial.py | Provides SpatialClusterer and GridCellAssigner for projected-coordinate spatial grouping. |
| app/pipeline/vnf/temporal.py | Calculates persistence, active months, active runs, gaps, trends, and persistent/transient labels. |
| app/pipeline/vnf/features.py | Aggregates detections into centroid, radius, thermal, area, radiant-heat, temporal, and fingerprint features. |
| app/pipeline/vnf/vnf_pipeline.py | Builds the reusable VNF processing and classification pipeline. |
| app/pipeline/__init__.py | Empty pipeline package initializer. |
| app/pipeline/vnf/__init__.py | Empty VNF pipeline package initializer. |

# 8. Services

## VNF services

| File | Purpose |
|---|---|
| app/services/vnf/vnf_ingestion_service.py | Normalizes VNF columns, validates data, removes duplicates, and bulk-inserts records. |
| app/services/vnf/vnf_incremental_service.py | Chooses bootstrap or incremental processing and coordinates the upload workflow. |
| app/services/vnf/vnf_pipeline_service.py | Runs the in-memory VNF pipeline and classifier, optionally preserving member detections. |
| app/services/vnf/vnf_candidate_matching_service.py | Matches candidate VNF detections to known thermal objects and updates state. |
| app/services/vnf/vnf_candidate_pipeline_service.py | Runs bootstrap and affected-history candidate processing. |
| app/services/vnf/vnf_candidate_state_service.py | Finalizes CANDIDATE, KNOWN_OBJECT, and TRANSIENT states and assigns object UIDs. |
| app/services/vnf/vnf_object_sync_service.py | Reuses matching objects or inserts new ThermalObject records. |
| app/services/vnf/vnf_object_profile_service.py | Recalculates affected object statistics and thermal fingerprints. |
| app/services/vnf/__init__.py | Empty VNF services package initializer. |

## FIRMS, matching, alerts, classification, and geography

| File | Purpose |
|---|---|
| app/services/firms/firms_api_service.py | Fetches India-bounding-box data from NASA FIRMS for VIIRS sources. |
| app/services/firms/firms_monitor_service.py | Processes individual FIRMS records and generates IDs, matches, and alerts. |
| app/services/firms/firms_batch_service.py | Performs bulk UID generation, deduplication, PostGIS matching, insertion, and alert creation. |
| app/services/firms/firms_runner_service.py | Orchestrates and records a complete FIRMS monitoring run. |
| app/services/firms/__init__.py | Empty FIRMS services package initializer. |

| File | Purpose |
|---|---|
| app/services/matching/matching_service.py | Finds the nearest thermal object within the matching radius, approximately 550 metres. |
| app/services/matching/batch_matching_service.py | Matches many FIRMS points using PostGIS JSON records, distance functions, and lateral joins. |
| app/services/matching/__init__.py | Empty matching services package initializer. |
| app/services/alerts/alert_service.py | Builds unmatched, repeated-unmatched, and abnormal-known-source alerts. |
| app/services/alerts/__init__.py | Empty alerts services package initializer. |
| app/services/classification/classifier_service.py | Loads the XGBoost artifact, prepares features, predicts labels, and returns confidence. |
| app/services/classification/__init__.py | Empty classification services package initializer. |
| app/services/geo/india_filter_service.py | Uses the India boundary polygon to filter FIRMS coordinates. |
| app/services/geo/__init__.py | Empty geographic services package initializer. |

# 9. Jobs and Data

## Jobs and data files

| File | Purpose |
|---|---|
| app/jobs/firms_monitoring_job.py | Runs run_firms_monitoring(day_range=1, trigger_type='SCHEDULED'). |
| app/jobs/__init__.py | Empty jobs package initializer. |
| app/data/raw/VNF_INDIA_2026.csv | Raw VNF satellite input data. |
| app/data/geo/india_boundary.geojson | India boundary geometry used for geographic filtering. |
| artifacts/PAN_INDIA_XGBOOST_THERMAL_VARIABILITY.pkl | Saved XGBoost thermal-variability classifier. |
| requirements.txt | Python dependencies required by the backend. |
| README.md | Currently contains only the project title and needs complete setup documentation. |

# 10. Tests and Utilities

## Test and maintenance scripts

| File | Purpose |
|---|---|
| tests/test_db.py | Checks database connectivity with SELECT 1. |
| tests/test_crud.py | Inserts and reads a sample ThermalObject. |
| tests/test_firms_api.py | Calls the NASA FIRMS API and prints returned data. |
| tests/test_firms_monitor.py | Processes one sample FIRMS record. |
| tests/test_firms_batch.py | Processes sample FIRMS records through the batch service. |
| tests/test_firms_runner.py | Runs a complete FIRMS monitoring cycle. |
| tests/test_match.py | Tests nearest thermal-object matching. |
| tests/test_persistent_match.py | Tests batch matching near an existing object. |

| File | Purpose |
|---|---|
| tests/test_unmatched_alert.py | Attempts unmatched-alert creation. |
| tests/test_vnf_ingestion.py | Loads local VNF data and tests ingestion and candidate matching. |
| tests/test_vnf_pipeline.py | Runs the historical VNF pipeline and synchronizes an object. |
| tests/test_vnf_pipeline_members.py | Runs the pipeline while preserving member detections. |
| tests/test_vnf_candidate_pipeline.py | Tests candidate pipeline processing. |
| tests/test_vnf_candidate_matching.py | Tests candidate-to-known-object matching. |
| tests/test_vnf_object_sync.py | Checks that an existing object is reused correctly. |
| tests/backfill_object_radius.py | Rebuilds pipeline objects and updates existing radii. |
| tests/__init__.py | Empty test package initializer. |

These files are mostly executable integration scripts rather than conventional pytest tests with test functions.

## 11. Known Maintenance Issues

- Some tests import function names that do not match the current service definitions.
- Some scripts expect VNF_INDIA_2026.csv at the repository root while the data file is under app/data/raw/.
- Some configured paths may refer to data/ instead of app/data/.
- The geographic boundary path should resolve to app/data/geo/india_boundary.geojson.
- The README should be expanded with installation, environment, database, API, and testing instructions.